
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 2

Introdução à Programação em C

Exercícios (2.7–2.31)

Resolvidos passo a passo, com código completo

Contents

1	Exercício 2.7 – Identificar e Corrigir Erros	2
2	Exercício 2.8 – Preencher Espaços	4
3	Exercício 2.9 – Instruções	4
4	Exercício 2.10 – Verdadeiro ou Falso	4
5	Exercício 2.11 – Lacunas	5
6	Exercício 2.12 – Saídas com $x = 2$ e $y = 3$	6
7	Exercício 2.13 – Substituição de Valores	6
8	Exercício 2.14 – Equação $y = ax^3 + 7$	6
9	Exercício 2.15 – Ordem de Avaliação	7
10	Exercício 2.16 – Aritmética Básica	7
11	Exercício 2.17 – Imprimir 1 a 4	9
12	Exercício 2.18 – Comparando Inteiros	9
13	Exercício 2.19 – Aritmética com 3 Inteiros	11
14	Exercício 2.20 – Círculo	12
15	Exercício 2.21 – Formas com Asteriscos	13
16	Exercício 2.22 – O que imprime?	13
17	Exercício 2.23 – Maior e Menor de 5 Inteiros	15
18	Exercício 2.24 – Par ou Ímpar	15
19	Exercício 2.25 – Iniciais em Bloco	17
20	Exercício 2.26 – Múltiplos	18
21	Exercício 2.27 – Padrão Alternado	18
22	Exercício 2.28 – Erros Fatais vs. Não Fatais	20
23	Exercício 2.29 – Valor Inteiro de Caracteres	20
24	Exercício 2.30 – Separar Dígitos	22
25	Exercício 2.31 – Tabela de Quadrados e Cubos	23

1. Exercício 2.7 – Identificar e Corrigir Erros

Resposta detalhada

a) `scanf( "d", valor );`

Erros: (1) falta % antes de d; (2) falta & antes de valor.

Correção:

```
1 scanf( "%d", &valor );
```

b) `printf( "0 produto de %d e %d e %d\n", x, y );`

Erros: (1) \n fora das aspas; (2) faltam argumentos para os 3 %d.

Correção:

```
1 printf( "0 produto de %d e %d e %d\n", x, y, x * y
    );
```

c) `primeiroNumero + segundoNumero = somaDosNumeros`

Erros: (1) atribuição com lado esquerdo inválido (expressão, não variável); (2) falta ;.

Correção:

```
1 somaDosNumeros = primeiroNumero + segundoNumero;
```

d) `if ( numero => maior ) maior == numero;`

Erros: (1) => deveria ser >; (2) == é comparação, não atribuição (deveria ser =).

Correção:

```
1 if ( numero >= maior ) {
2     maior = numero;
3 }
```

e) `*/ Programa para determinar o maior dentre tres inteiros /*`

Erro: delimitadores de comentário invertidos. O correto é abrir com /* e fechar com */.

Correção:

```
1 /* Programa para determinar o maior dentre tres
    inteiros */
```

f) `Scanf( "%d", umInteiro );`

Erros: (1) Scanf com S maiúsculo (C é case-sensitive); (2) falta &.

Correção:

```
1 scanf( "%d", &umInteiro );
```

g) `printf( "Modulo de %d dividido por %d e\n", x, y, x % y );`

Erro: faltam especificadores. Há 3 argumentos mas só 2 %d.

Correção:

```
1 printf( "Modulo de %d dividido por %d e %d\n", x, y
    , x % y );
```

- h) `if ( x = y ); printf( %d e igual a %d\n", x, y );`
Erros: (1) `=` é atribuição (deveria ser `==`); (2) `;` após condição; (3) falta aspas iniciais em `printf`.

Correção:

```
1  if ( x == y ) {  
2      printf( "%d e igual a %d\n", x, y );  
3  }
```

- i) `print( "A soma e %d\n," x + y );`
Erros: (1) `print` (faltou o `f`); (2) vírgula *dentro* das aspas em vez de fora.

Correção:

```
1  printf( "A soma e %d\n", x + y );
```

- j) `Printf( "O valor que voce digitou e: %d\n, &valor );`
Erros: (1) `Printf` com `P` maiúsculo; (2) aspas duplas não fechadas; (3) `&` desnecessário em `printf`.

Correção:

```
1  printf( "O valor que voce digitou e: %d\n", valor )  
    ;
```

2. Exercício 2.8 – Preencher Espaços

Resposta detalhada

- a) **Comentários** são usados para documentar e melhorar a legibilidade.
- b) A função `printf` exibe informações na tela.
- c) Instrução de tomada de decisão: `if`.
- d) Cálculos são realizados por instruções de **atribuição** (com operadores aritméticos).
- e) A função `scanf` lê valores do teclado.

3. Exercício 2.9 – Instruções

Resposta detalhada

- a) Exibir mensagem:

```
1 printf( "Digite dois numeros.\n" );
```

- b) Atribuir produto:

```
1 a = b * c;
```

- c) Comentário documentando:

```
1 /* Calculo de folha de pagamento */
```

- d) Ler 3 inteiros:

```
1 scanf( "%d%d%d", &a, &b, &c );
```

4. Exercício 2.10 – Verdadeiro ou Falso

Resposta detalhada

- a) **Falso.** Os operadores aritméticos em C *associam* da esquerda para a direita **no mesmo nível de precedência**, mas a *avaliação* respeita a hierarquia de precedência (multiplicação antes de adição, por exemplo).
- b) **Falso.** `_sub_barra_` é válido, mas vários dos nomes têm acentos (e nomes com acentos podem ou não ser aceitos dependendo do compilador/locale). Em C tradicional, identificadores devem usar apenas letras ASCII (a-z, A-Z), dígitos e `_`, e não começar com dígito. Os exemplos sem acentos como `m928134`, `t5`, `j7`, `a`, `b`, `c`, `z`, `z2` são válidos.
- c) **Falso.** `printf("a = 5;");` é uma chamada de função que *exibe na tela* a string "a = 5;" – não é uma atribuição (que seria `a = 5;` sem aspas).
- d) **Falso.** Avaliação respeita a precedência dos operadores. Em `2 + 3 * 4`, a multiplicação ocorre primeiro, dando 14 – não da esquerda para a direita (que daria 20).
- e) **Verdadeiro.** Identificadores em C não podem começar com dígito. `3g`,

87, 67h2, 2h começam com dígito (inválidos). h22 é válido. (Os números puros nem são identificadores.)

5. Exercício 2.11 – Lacunas

Resposta detalhada

- a) Operações no mesmo nível da multiplicação: **divisão (/) e módulo (%)**.
- b) Em parênteses aninhados, o **conjunto mais interno** é avaliado primeiro.
- c) Uma posição na memória que pode conter diferentes valores ao longo do tempo é chamada de **variável**.

6. Exercício 2.12 – Saídas com $x = 2$ e $y = 3$

Resposta detalhada

- a) `printf( "%d", x );` → **2**
- b) `printf( "%d", x + x );` → **4**
- c) `printf( "x=" );` → **x=**
- d) `printf( "x=%d", x );` → **x=2**
- e) `printf( "%d = %d", x + y, y + x );` → **5 = 5**
- f) `z = x + y;` → **Nada** (atribuição não imprime)
- g) `scanf( "%d%d", &x, &y );` → **Nada** (entrada, não saída)
- h) `/* printf("x + y = %d", x + y); */` → **Nada** (comentário)
- i) `printf( "\n" );` → **Nova linha** (caractere de quebra)

7. Exercício 2.13 – Substituição de Valores

Resposta detalhada

Quais instruções modificam variáveis?

- a) `scanf("%d%d%d%d%d", &b, &c, &d, &e, &f);` – **SIM**, modifica b, c, d, e, f.
- b) `p = i + j + k + 7;` – **SIM**, modifica p.
- c) `printf("Valores sao substituidos");` – **NÃO**, apenas exibe.
- d) `printf("a = 5");` – **NÃO**, apenas exibe (a string “a = 5”, não uma atribuição!).

Boa prática de programação

`printf` é apenas saída – nunca modifica nada. `scanf` e as atribuições (`x = ...`) são as principais formas de modificar variáveis neste capítulo.

8. Exercício 2.14 – Equação $y = ax^3 + 7$

Resposta detalhada

- a) `y = a * x * x * x + 7;` – **CORRETO**. $a \cdot x \cdot x \cdot x + 7 = ax^3 + 7$.
- b) `y = a * x * x * ( x + 7 );` – **INCORRETO**. Calcula $a \cdot x^2 \cdot (x+7) = ax^3 + 7ax^2$.
- c) `y = ( a * x ) * x * ( x + 7 );` – **INCORRETO**. Equivalente a (b): $ax \cdot x \cdot (x+7) = ax^3 + 7ax^2$.
- d) `y = ( a * x ) * x * x + 7;` – **CORRETO**. $ax \cdot x \cdot x + 7 = ax^3 + 7$.
- e) `y = a * ( x * x * x ) + 7;` – **CORRETO**. $a \cdot x^3 + 7$.
- f) `y = a * x * ( x * x + 7 );` – **INCORRETO**. Calcula $ax \cdot (x^2+7) = ax^3 + 7ax$.

Resposta: (a), (d) e (e) estão corretos.

9. Exercício 2.15 – Ordem de Avaliação

Resposta detalhada

a) $x = 7 + 3 * 6 / 2 - 1$;

Precedência: *, / (mesmo nível, esq. para dir.), depois +, -.

1. $3 * 6 = 18$

2. $18 / 2 = 9$

3. $7 + 9 = 16$

4. $16 - 1 = 15$

$x = 15$.

b) $x = 2 \% 2 + 2 * 2 - 2 / 2$;

1. $2 \% 2 = 0$

2. $2 * 2 = 4$

3. $2 / 2 = 1$

4. $0 + 4 = 4$

5. $4 - 1 = 3$

$x = 3$.

c) $x = (3 * 9 * (3 + (9 * 3 / (3))))$;

Parênteses mais internos primeiro:

1. $(3) = 3$

2. $9 * 3 = 27$

3. $27 / 3 = 9$

4. $3 + 9 = 12$

5. $3 * 9 = 27$

6. $27 * 12 = 324$

$x = 324$.

10. Exercício 2.16 – Aritmética Básica

Resposta detalhada

```
1  /* Ex2.16: aritmetica.c
2     Le dois numeros e exhibe varias operacoes */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     int a, b;
8
9     printf( "Digite dois inteiros: " );
10    scanf( "%d%d", &a, &b );
11
12    printf( "Soma:      %d + %d = %d\n", a, b, a + b )
13    ;
14    printf( "Produto:   %d * %d = %d\n", a, b, a * b )
15    ;
```



```
14     printf( "Diferença: %d - %d = %d\n", a, b, a - b )
      ;
15     printf( "Quociente: %d / %d = %d\n", a, b, a / b )
      ;
16     printf( "Modulo:      %d %% %d = %d\n", a, b, a % b
      );
17
18     return 0;
19 }
```

Atenção: se o usuário digitar 0 para b, haverá divisão por zero em a / b e $a \% b$ – comportamento indefinido (geralmente, falha do programa). Nos capítulos seguintes aprenderemos a validar a entrada.

11. Exercício 2.17 – Imprimir 1 a 4

Resposta detalhada

(a) Sem especificadores de conversão:

```
1 printf( "1 2 3 4\n" );
```

(b) Com 4 especificadores:

```
1 printf( "%d %d %d %d\n", 1, 2, 3, 4 );
```

(c) Quatro printf:

```
1 printf( "%d ", 1 );  
2 printf( "%d ", 2 );  
3 printf( "%d ", 3 );  
4 printf( "%d\n", 4 );
```

Os três produzem a mesma saída: 1 2 3 4.

12. Exercício 2.18 – Comparando Inteiros

Resposta detalhada

```
1  /* Ex2.18: maior_dois.c */  
2  #include <stdio.h>  
3  
4  int main( void )  
5  {  
6      int a, b;  
7  
8      printf( "Digite dois inteiros: " );  
9      scanf( "%d%d", &a, &b );  
10  
11     if ( a > b ) {  
12         printf( "%d e maior\n", a );  
13     }  
14     if ( b > a ) {  
15         printf( "%d e maior\n", b );  
16     }  
17     if ( a == b ) {  
18         printf( "Esses numeros sao iguais\n" );  
19     }  
20  
21     return 0;  
22 }
```

Por que três if's separados? O enunciado pede para usar *apenas a forma de seleção única* – não temos `else` ainda (será visto no Cap. 3). Cada caso é testado com um `if` independente.

Como as condições são mutuamente exclusivas, exatamente um dos `if's` será

verdadeiro em qualquer execução.

13. Exercício 2.19 – Aritmética com 3 Inteiros

Resposta detalhada

```
1  /* Ex2.19: tres_inteiros.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int a, b, c;
7      int soma, produto, menor, maior;
8      int media;
9
10     printf( "Digite tres inteiros diferentes: " );
11     scanf( "%d%d%d", &a, &b, &c );
12
13     soma = a + b + c;
14     media = soma / 3;  /* divisao inteira -- arredonda
15                        para baixo */
16     produto = a * b * c;
17
18     /* Comeca assumindo que 'a' e o menor e o maior */
19     menor = a;
20     maior = a;
21
22     /* Testa b */
23     if ( b < menor ) menor = b;
24     if ( b > maior ) maior = b;
25
26     /* Testa c */
27     if ( c < menor ) menor = c;
28     if ( c > maior ) maior = c;
29
30     printf( "A soma e %d\n", soma );
31     printf( "A media e %d\n", media );
32     printf( "O produto e %d\n", produto );
33     printf( "O menor e %d\n", menor );
34     printf( "O maior e %d\n", maior );
35
36     return 0;
37 }
```

Exemplo (entrada 13 27 14):

Saída do programa

```
A soma e 54
A media e 18
O produto e 4914
O menor e 13
O maior e 27
```

Nota: a média é calculada com divisão inteira (resultado sem casas decimais). $54/3 = 18$, exato. Mas para $14/3 = 4$ (não 4,67), pois truncamos no inteiro.

14. Exercício 2.20 – Círculo

Resposta detalhada

```
1  /* Ex2.20: circulo.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int raio;
7
8      printf( "Digite o raio: " );
9      scanf( "%d", &raio );
10
11     printf( "Diametro:      %f\n", raio * 2.0 );
12     printf( "Circunferencia: %f\n", 2 * 3.14159 * raio
13             );
14     printf( "Area:          %f\n", 3.14159 * raio *
15             raio );
16
17     return 0;
18 }
```

Observação importante: como raio é int, mas usamos constantes double (2.0, 3.14159), o C *promove* o resultado para ponto flutuante automaticamente. Por isso, %f funciona mesmo sem variáveis double explícitas. Isso é discutido com mais detalhes no Cap. 3.

15. Exercício 2.21 – Formas com Asteriscos

Resposta detalhada

```

1  /* Ex2.21: formas_asteriscos.c
2     Imprime quatro formas: quadrado, oval, seta,
3     diamante */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9
10     /* Cada printf corresponde a uma linha das quatro
11        formas
12        lado a lado: quadrado, oval, seta, diamante */
13
14     printf( "*****   ***   *   *\n" );
15     printf( "*       * *   *   ***   *\n" );
16     printf( "*       * *   *   ***** *\n" );
17     printf( "*       * *   *   *   *****\n" );
18     printf( "*       * *   *   *   *\n" );
19     printf( "*       * *   *   *   *\n" );
20     printf( "*       * *   *   *   *\n" );
21     printf( "*       * *   *   *   *\n" );
22     printf( "*****   ***   *   *\n" );
23
24     return 0;
25 }

```

Observação: as formas exatas do livro variam um pouco; o importante é que cada `printf` imprima uma linha completa contendo a fatia horizontal de todas as figuras. Exercite seu senso de arte ASCII!

16. Exercício 2.22 – O que imprime?

Resposta detalhada

Código:

```

1  printf( "*\n**\n***\n****\n*****\n" );

```

Saída:

Saída do programa

```

*
**
***
****
*****

```

Um triângulo de asteriscos, com cada `\n` forçando uma quebra de linha. Belo

exemplo de como uma única chamada a `printf` pode produzir múltiplas linhas.

17. Exercício 2.23 – Maior e Menor de 5 Inteiros

Resposta detalhada

```
1  /* Ex2.23: extremos_cinco.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int a, b, c, d, e;
7      int menor, maior;
8
9      printf( "Digite 5 inteiros: " );
10     scanf( "%d%d%d%d%d", &a, &b, &c, &d, &e );
11
12     menor = a;
13     maior = a;
14
15     if ( b < menor ) menor = b;
16     if ( b > maior ) maior = b;
17     if ( c < menor ) menor = c;
18     if ( c > maior ) maior = c;
19     if ( d < menor ) menor = d;
20     if ( d > maior ) maior = d;
21     if ( e < menor ) menor = e;
22     if ( e > maior ) maior = e;
23
24     printf( "Menor: %d\n", menor );
25     printf( "Maior: %d\n", maior );
26
27     return 0;
28 }
```

Nota: este padrão de “inicializar com o primeiro elemento, depois comparar com os demais” é a base de algoritmos de busca em arrays (Cap. 6).

18. Exercício 2.24 – Par ou Ímpar

Resposta detalhada

```
1  /* Ex2.24: par_impar.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int n;
7
8      printf( "Digite um inteiro: " );
9      scanf( "%d", &n );
```



```
10
11     if ( n % 2 == 0 ) {
12         printf( "%d e par\n", n );
13     }
14     if ( n % 2 != 0 ) {
15         printf( "%d e impar\n", n );
16     }
17
18     return 0;
19 }
```

Conceito-chave: o operador módulo % retorna o resto da divisão. $n \% 2$ é 0 se n é par, 1 (ou -1 para negativos) se ímpar. Essa é uma técnica fundamental.

19. Exercício 2.25 – Iniciais em Bloco

Resposta detalhada

Exemplo para as iniciais PJD (Paul J. Deitel):

```

1  /* Ex2.25: iniciais.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      /* Letra P */
7      printf( "PPPPPPPP\n" );
8      printf( "P      P\n" );
9      printf( "P      P\n" );
10     printf( "P      P\n" );
11     printf( "PPPPPPPP\n" );
12     printf( "P\n" );
13     printf( "P\n" );
14     printf( "P\n" );
15     printf( "P\n\n" );
16
17     /* Letra J */
18     printf( "          JJ\n" );
19     printf( "          J\n" );
20     printf( "          J\n" );
21     printf( "          J\n" );
22     printf( "          J\n" );
23     printf( "          J\n" );
24     printf( "J          J\n" );
25     printf( "J          J\n" );
26     printf( "JJJJJJJJ\n\n" );
27
28     /* Letra D */
29     printf( "DDDDDDDD\n" );
30     printf( "D          D\n" );
31     printf( "D          D\n" );
32     printf( "D          D\n" );
33     printf( "D          D\n" );
34     printf( "D          D\n" );
35     printf( "D          D\n" );
36     printf( "D          D\n" );
37     printf( "DDDDDDDD\n" );
38
39     return 0;
40 }
```

Adapte para suas próprias iniciais! A ideia é treinar formatação com `printf` e atenção a detalhes visuais.

20. Exercício 2.26 – Múltiplos

Resposta detalhada

```

1  /* Ex2.26: multiplos.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int a, b;
7
8      printf( "Digite dois inteiros: " );
9      scanf( "%d%d", &a, &b );
10
11     if ( a % b == 0 ) {
12         printf( "%d e multiplo de %d\n", a, b );
13     }
14     if ( a % b != 0 ) {
15         printf( "%d NAO e multiplo de %d\n", a, b );
16     }
17
18     return 0;
19 }

```

Lógica: se o resto da divisão de a por b é zero, então a é múltiplo de b.

Atenção: se b == 0, há divisão por zero. Em programas robustos, sempre valide a entrada.

21. Exercício 2.27 – Padrão Alternado

Resposta detalhada

Versão com 8 printf:

```

1  printf( "* * * * * * * *\n" );
2  printf( " * * * * * * * *\n" );
3  printf( "* * * * * * * *\n" );
4  printf( " * * * * * * * *\n" );
5  printf( "* * * * * * * *\n" );
6  printf( " * * * * * * * *\n" );
7  printf( "* * * * * * * *\n" );
8  printf( " * * * * * * * *\n" );

```

Versão com 1 printf:

```

1  printf( "* * * * * * * *\n * * * * * * * *\n"
2         " * * * * * * * *\n * * * * * * * *\n"
3         " * * * * * * * *\n * * * * * * * *\n"
4         " * * * * * * * *\n * * * * * * * *\n" );

```

Truque: em C, duas strings literais adjacentes (separadas só por

espaço/quebra) são automaticamente *concatenadas* pelo compilador. "abc"
"def" é equivalente a "abcdef".

22. Exercício 2.28 – Erros Fatais vs. Não Fatais

Resposta detalhada

Erro fatal: causa o encerramento *imediato* do programa, geralmente com falha de segmentação ou mensagem de erro do sistema operacional. Exemplos: divisão por zero, acesso a memória inválida (Cap. 7).

Erro não fatal: o programa *continua executando*, mas produz resultados incorretos. Exemplos: lógica errada num `if`, fórmula matemática incorreta, = em vez de ==.

Por que erro fatal é “preferível”? Um erro fatal é **óbvio** – o programa para, você sabe que há um problema, e geralmente o sistema indica onde foi o erro. Um erro não fatal pode **passar despercebido** por meses ou anos, gerando dados corrompidos, decisões erradas e bugs difíceis de rastrear.

Boa prática de programação

Erros silenciosos são os mais perigosos. Sempre que possível, prefira soluções que *falham cedo e ruidosamente* a soluções que escondem problemas. O uso de `assert()` (que estudaremos mais adiante) ajuda muito nisso.

23. Exercício 2.29 – Valor Inteiro de Caracteres

Resposta detalhada

```
1  /* Ex2.29: ascii.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      printf( "A = %d\n", 'A' );
7      printf( "B = %d\n", 'B' );
8      printf( "C = %d\n", 'C' );
9      printf( "a = %d\n", 'a' );
10     printf( "b = %d\n", 'b' );
11     printf( "c = %d\n", 'c' );
12     printf( "0 = %d\n", '0' );
13     printf( "1 = %d\n", '1' );
14     printf( "2 = %d\n", '2' );
15     printf( "$ = %d\n", '$' );
16     printf( "*" = %d\n", '*' );
17     printf( "+" = %d\n", '+' );
18     printf( "/" = %d\n", '/' );
19     printf( "(espaco) = %d\n", ' ' );
20
21     return 0;
22 }
```

Saída esperada (ASCII):**Saída do programa**

```
A = 65
B = 66
C = 67
a = 97
b = 98
c = 99
0 = 48
1 = 49
2 = 50
$ = 36
= 42
+ = 43
/ = 47
(espaco) = 32
```

Observações importantes:

- 'A' a 'Z': 65 a 90 (consecutivos).
- 'a' a 'z': 97 a 122 (também consecutivos, 32 a mais que maiúsculas).
- '0' a '9': 48 a 57.
- Para converter dígito ASCII em valor: '5' - '0' == 5.
- Para converter minúscula em maiúscula: 'a' - 32 == 'A'.

Esses truques aparecerão muito nos Caps. 6 a 8.

24. Exercício 2.30 – Separar Dígitos

Resposta detalhada

```
1  /* Ex2.30: separa_digitos.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int n;
7      int d1, d2, d3, d4, d5;
8
9      printf( "Digite um numero de 5 digitos: " );
10     scanf( "%d", &n );
11
12     d1 = n / 10000;           /* dezenas de milhar */
13     d2 = ( n / 1000 ) % 10;  /* milhar
14                               */
15     d3 = ( n / 100 ) % 10;   /* centena
16                               */
17     d4 = ( n / 10 ) % 10;    /* dezena
18                               */
19     d5 = n % 10;             /* unidade
20                               */
21
22     printf( "%d %d %d %d %d\n", d1, d2, d3, d4
23           , d5 );
24
25     return 0;
26 }
```

Trace para 42139:

- $42139 / 10000 = 4$
- $42139 / 1000 \% 10 = 42 \% 10 = 2$
- $42139 / 100 \% 10 = 421 \% 10 = 1$
- $42139 / 10 \% 10 = 4213 \% 10 = 3$
- $42139 \% 10 = 9$

Saída: 4 2 1 3 9

Lições: dois operadores aritméticos – divisão inteira (/) e módulo (%) – são suficientes para extrair qualquer dígito de um número inteiro. Esse padrão é usado em conversões de base, formatação numérica, etc.

25. Exercício 2.31 – Tabela de Quadrados e Cubos

Resposta detalhada

```
1  /* Ex2.31: quadrados_cubos.c */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      printf( "numero\tquadrado\tcubo\n" );
7      printf( "0\t0\t\t0\n" );
8      printf( "1\t1\t\t1\n" );
9      printf( "2\t4\t\t8\n" );
10     printf( "3\t9\t\t27\n" );
11     printf( "4\t16\t\t64\n" );
12     printf( "5\t25\t\t125\n" );
13     printf( "6\t36\t\t216\n" );
14     printf( "7\t49\t\t343\n" );
15     printf( "8\t64\t\t512\n" );
16     printf( "9\t81\t\t729\n" );
17     printf( "10\t100\t\t1000\n" );
18
19     return 0;
20 }
```

Saída:

Saída do programa

```
numero  quadrado  cubo
0       0         0
1       1         1
2       4         8
...
10      100      1000
```

Versão alternativa calculando: aproveitando os operadores aritméticos:

```
1  int i = 0;
2  printf( "%d\t%d\t\t%d\n", i, i * i, i * i * i );
3  i = 1;
4  printf( "%d\t%d\t\t%d\n", i, i * i, i * i * i );
5  /* ... etc ... */
```

Nota: é repetitivo escrever 11 linhas explicitamente! No Capítulo 3 aprenderemos `while` – um único loop substituirá essas linhas todas.